গভীর ডেক · D2

স্ট্যাক ও সংযোগ

Python 3.13 · Django 6.1.2 · PostgreSQL · ৪টি settings মডিউল · ১০টি মিডলওয়্যার · ৮৫টি অ্যাপ রুট — একটি রিকোয়েস্ট কোথা দিয়ে যায়

মানচিত্র

এই ২৮ স্লাইডে কী আছে

চারটি ব্লক — স্ট্যাক থেকে শুরু, রিকোয়েস্টের পথ ধরে শেষ

১. স্ট্যাক

স্লাইড ৩–৬ · ৯টি প্যাকেজ · দুই requirements · কেন কোনো React/Celery/Redis নেই

প্রমাণ: pinned ভার্সন

২. কনফিগারেশন

স্লাইড ৭–১০ · ৪টি settings মডিউল · env() · prod-এর দুই গার্ড

প্রমাণ: RuntimeError আউটপুট

৩. রিকোয়েস্টের পথ

স্লাইড ১১–১৮ · ১০ মিডলওয়্যার · ৮৫ রুট · ৫২টি সাজানো ভিউ

প্রমাণ: রুট-গণনা

৪. পর্দা ও পরিবহন

স্লাইড ১৯–২৮ · কনটেক্সট · ৭৫ টেমপ্লেট · ৫ স্ট্যাটিক ফাইল · সেশন · কমান্ড · ফাঁক

প্রমাণ: ফাইল-গণনা

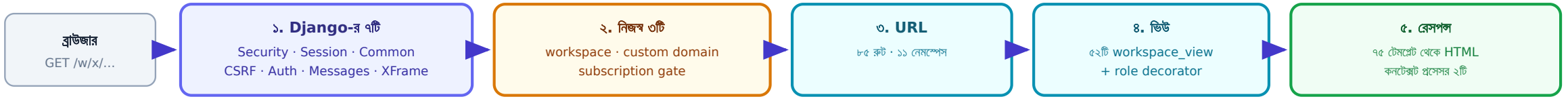
এই ডেকের প্রতিটি সংখ্যা `pip list`, `get_resolver()` আর `find` দিয়ে গোনা — কোনো স্লাইডে অনুমান নেই।

শিক্ষকের নোট: বলুন এই ডেকটি "চার্ট" নয়, "নকশা" — প্রতিটি সেটিং কেন সেখানে, সেটাই মূল প্রশ্ন। শিক্ষার্থীদের নিজের প্রজেক্টের `settings.py` খুলে একই প্রশ্ন করতে বলুন।

পথ

একটি GET রিকোয়েস্টের পুরো যাত্রা

ব্রাউজার থেকে পর্দা পর্যন্ত — ৫টি ধাপ, ১০টি দরজা



মিডলওয়্যারে যা তৈরি হয়

request.workspace · request.membership
request.branding · request.domain_workspace
ভিউ শরীরে আর কোনো লুকআপ নেই

ভিউ যা করে

scoping.py → দৃশ্যমান জিনিসের কোয়েরি
forms.validation → লিখবে কি না
৫২টি ভিউতে একই দুই প্রশ্ন

টেমপ্লেট যা পায়

nav_groups · brand · product · subscription
কনটেক্সট প্রসেসর থেকেই, প্রতি পাতায়
কোনো ভিউ আলাদা করে পাঠায় না

ক্রমটাই নিরাপত্তা: Auth মিডলওয়্যার আগে থাকায় আমাদের workspace মিডলওয়্যার request.user পায়

আর SubscriptionGate সবার পরে — কারণ সে request.workspace-এর উপর নির্ভর করে, যা তার আগের মিডলওয়্যার বানিয়েছে

ক্রম বদলালে ভেঙে যায়, কিন্তু এরর আসে না — শুধু কিছু রিকোয়েস্ট চুপচাপ ভুল পথে যায়

শিক্ষকের নোট: বোর্ডে শুধু "10 → 85 → 52" লিখুন। তারপর জিজ্ঞেস করুন: "মিডলওয়্যার আর ভিউ — কে কী জানে?" মিডলওয়্যার জানে "কে, কোন স্কুলে"; ভিউ জানে "কী দেখাবে"।

স্ট্যাক

৯টি প্যাকেজ — প্রতিটির একটি কাজ

সবগুলো ভার্সন পিন করা · `requirements.txt` + `requirements-prod.txt`

প্যাকেজ	কী কাজে
`Django==6.1.2`	ওয়েব ফ্রেমওয়ার্ক · ORM · টেমপ্লেট · admin
`psycopg[binary]==3.3.6`	PostgreSQL ড্রাইভার (v3, binary wheel)
`python-dotenv==1.2.4`	`.env` পড়া — কোডে সিক্রেট নয়
`Pillow==12.3.0`	লোগো/ছবি যাচাই (`ImageField`)
`openpyxl==3.1.5`	CSV/Excel এক্সপোর্ট
`reportlab==5.1.0`	রেজাল্ট-শিট PDF

শুধু প্রোডাকশনে	কী কাজে
`gunicorn==23.0.0`	WSGI সার্ভার — `runserver` নয়
`whitenoise==6.11.0`	স্ট্যাটিক ফাইল অ্যাপ থেকেই সার্ভ
`dj-database-url==3.0.1`	`DATABASE_URL` পার্সার — **এখনো ব্যবহৃত হয়নি**

সৎ ফাঁক: `dj-database-url` ইনস্টল করা আছে আর `.env.example`-এ `DATABASE_URL` লেখা আছে, কিন্তু কোডে কোথাও তা পড়া হয় না — `base.py` আলাদা `DB_NAME/DB_USER/…` ব্যবহার করে।

সব ভার্সন **ঠিক** লেখা (`==`), `>=` নয়। কারণ: "কাল অন্য ভার্সনে চলে গেল" — এই ব্যাখ্যা কোনো ডেমোতে কেউ মানে না।

শিক্ষকের নোট: শেষ বাক্যটি জোর দিন। শিক্ষার্থীদের অভ্যাস থাকে `pip install django` আর `pip freeze` না করা — ছয় মাস পরে নিজের প্রজেক্টই আর চলে না।

সিদ্ধান্ত

যা ইচ্ছাকৃতভাবে রাখা হয়নি

প্রতিটি বাদ দেওয়া একটা সিদ্ধান্ত — আর একটা সীমা

যা নেই	কেন বাদ	কখন দরকার হবে
React / SPA	২ ভূমিকা, ৮৫ রুট, ফর্ম-ভারী পর্দা — server-rendered HTML যথেষ্ট; একটি বিল্ড-ধাপ কম মানে একটি ভাঙার জায়গা কম	যখন অফলাইন বা রিয়েল-টাইম হাজিরা দরকার হবে
Celery / Redis	এখন কোনো কাজই ব্যাকগ্রাউন্ডে চলে না; PDF ১ সেকেন্ডে হয়	ইমেইল-বাল্ক, রিপোর্ট-জেনারেশন, রিমাইন্ডার
Docker	ডেমোটি `bash run.sh` — শিক্ষার্থীর মেশিনে Docker মানে আরও একটি ইনস্টল	দল বড় হলে, বা সার্ভার একাধিক হলে
GraphQL / REST API	এখন কনজিউমার কেবল ব্রাউজার; API বানালে সেটি রক্ষণাবেক্ষণের আরও একটি চুক্তি	মোবাইল অ্যাপ এলে (D2-এর বাইরে, ভবিষ্যৎ ডেকে)
মাইক্রোসার্ভিস	একই ডেটাবেজ, একই ট্রানজেকশন — ভাগ করলে JOIN-ই HTTP কল হয়ে যেত	দল ২০+ হলে, আর ভিন্ন স্কেলিং দরকার হলে
ক্যাশ (Memcached)	ডেটা ছোট, কোয়েরি indexed; ক্যাশ যোগ করলে "পুরনো ডেটা" বাগ আসে	ড্যাশবোর্ড ধীর হলে — মেপে দেখে

নিয়ম: **প্রতিটি বাদ দেওয়া জিনিসের জন্য একটি লিখিত শর্ত** — "কখন যোগ করব"। শর্ত ছাড়া বাদ দেওয়া মানে ভুলে যাওয়া।

শিক্ষকের নোট: শিক্ষার্থীরা প্রায়ই জিজ্ঞেস করে "React ছাড়া চলে?" — উত্তর: চলে, এবং এখানে সেটাই সঠিক সিদ্ধান্ত, কারণ সমস্যাটি UI-এর জটিলতা নয়, ডেটার নিয়মের জটিলতা। প্রতিটি সারিতে "কখন দরকার" কলামটি তুলে ধরুন।

দুই ফাইল

`requirements.txt` বনাম `requirements-prod.txt`

৬ + ৪ = ১০টি লাইন, কিন্তু সিদ্ধান্ত বড়

```
# requirements.txt  (দুই জায়গায়)
Django==6.1.2
psycopg[binary]==3.3.6
python-dotenv==1.2.4
Pillow==12.3.0
openpyxl==3.1.5
reportlab==5.1.0
```

```
# requirements-prod.txt
-r requirements.txt
gunicorn==23.0.0
whitenoise==6.11.0
dj-database-url==3.0.1
```

ডেমো, টেস্ট, ডেভেলপমেন্ট — সবখানে একই।

প্রথম লাইনটাই মূল: `-r` — base তালিকা আবার লিখতে হয় না, তাই দুটো কখনো আলাদা হয়ে যায় না।

প্রশ্ন	উত্তর
কেন gunicorn বেসে নেই?	শিক্ষার্থীর মেশিনে `runserver` চলে; gunicorn ইনস্টল থাকলে কেউ ভুল করে সেটি ডেমোতে চালাত
কেন whitenoise আলাদা?	ডেভে `runserver` নিজেই স্ট্যাটিক দেয়; প্রোডে সেই সুবিধা নেই
`pip freeze` চলে?	না — হাতে লেখা, তাই কোনটি সরাসরি দরকার আর কোনটি টানল, তা পরিষ্কার

শিক্ষকের নোট: ছোট প্রশ্ন: "prod ফাইলে `-r` না লিখে ৬টি লাইন কপি করলে কী হয়?" উত্তর: একদিন Django ভার্সন এক জায়গায় বাড়বে, অন্যটায় থাকবে — আর প্রোডে অচেনা বাগ।

কনফিগ

একটি `settings.py` নয় — চারটি মডিউল, ৩০০ লাইন

`config/settings/` · base ১৮৯ · dev ১৪ · prod ৭৪ · sqlite ২৩

মডিউল	লাইন	কী থাকে	কী বদলায়
base.py	১৮৯	সব সাধারণ সেটিং: ১১টি অ্যাপ, ১০টি মিডলওয়্যার, টেমপ্লেট, DB, i18n, লগিং	কিছুই না — এখানে কোনো গোপন মান নেই
dev.py	১৪	`DEBUG=True`, `ALLOWED_HOSTS=["*"]`	কনসোল ইমেইল, লোকাল CSRF
sqlite.py	২৩	শুধু ডেটাবেজ বদল — PostgreSQL নয়, ফাইল	শূন্য-সেটআপ ডেমোর জন্য
prod.py	৭৪	HTTPS, HSTS, cookie-র flags, WhiteNoise, SMTP, লগিং	নিরাপত্তার সব কঠোর মান

প্রতিটি ফাইলের প্রথম লাইন এক: `from .base import *`। তাই base-এ কিছু যোগ করলে চার জায়গায় পাওয়া যায় — কপি করে রাখতে হয় না।

শৃঙ্খলা: `dev.py` কখনো prod-এ ব্যবহার হয় না, `sqlite.py` কখনো ডিপ্লয়ে হয় না। কোনটি চলছে, তা `DJANGO_SETTINGS_MODULE` বলে।

শিক্ষকের নোট: প্রশ্ন করুন: "সব এক ফাইলে রাখলে কী হারাতাম?" উত্তর: নিরাপত্তা-মানগুলো আলাদা করে দেখার সুযোগ। prod.py খুললেই বোঝা যায় প্রোডাকশনে কী কী কঠোর নিয়ম আছে।

গোপন

এক ফাংশন `env()`, আর সিক্রেট কোথাও নেই

base.py-তে একটি পাসওয়ার্ডও লেখা নেই — সব `.env` থেকে

```
def env(key, default=None):
    value = os.environ.get(key, default)
    if value is None:
        raise RuntimeError(
            f"Missing required environment variable: {key}. "
            f"Copy .env.example to .env and fill it in.")
    return value

SECRET_KEY = env("SECRET_KEY", DEV_INSECURE_KEY)
DEBUG = env_bool("DEBUG", False)
ALLOWED_HOSTS = env_list("ALLOWED_HOSTS", "127.0.0.1,localhost")
```

ফাংশন	ব্যবহার
`env()`	স্ট্রিং; না পেলে স্পষ্ট এরর, নীরব None নয়
`env_bool()`	`"1/true/yes/on"` → True
`env_list()`	কমা দিয়ে ভাগ করা তালিকা

কেন স্পষ্ট এরর? `os.environ.get()` চুপচাপ `None` দিলে অ্যাপ চালু হয়, তারপর অর্ধেক কাজ করে — ভুলটা তখন অনেক দূরে খুঁজতে হয়।

.env.example-এ ৫টি কী — কিন্তু কোডে পড়া হয় ৪টির বেশি: `DB_NAME/DB_USER/DB_PASSWORD/DB_HOST/DB_PORT` ডকুমেন্টেড নয়। নতুন কেউ সেটআপ করতে গেলে অনুমান করতে হয়।

শিক্ষকের নোট: `.env` কখনো Git-এ যায় না — `.gitignore`-এ আছে। আর `.env.example` যায়, কারণ তাতে কোনো আসল মান নেই। এই দুই ফাইলের পার্থক্যটাই নতুন ডেভেলপারের প্রথম পাঠ।

গার্ড

prod.py চালুই হবে না — দুটি ভুল সেটিং পেলে

সেটিং "ঢিলা করে" ডিপ্লয় ঠিক করার সংস্কৃতি বন্ধ

```
$ SECRET_KEY=dev-insecure-key-... \
  DJANGO_SETTINGS_MODULE=config.settings.prod \
  python -c "import django; django.setup()"

RuntimeError: Refusing to start: SECRET_KEY is the
development default. Generate one with:
python -c "import secrets; print(
    secrets.token_urlsafe(50))"
```

```
$ ALLOWED_HOSTS='*' ... django.setup()

RuntimeError: ALLOWED_HOSTS must list real
hostnames in production; '*' is not allowed.
```

সঠিক সেটিং দিয়ে Django-র নিজের পরীক্ষা

```
$ SECRET_KEY=$(python -c "import secrets;
    print(secrets.token_urlsafe(50))")
$ ALLOWED_HOSTS=ims.example.com \
  python manage.py check --deploy

System check identified no issues
(0 silenced).
```

ফল: `check --deploy`-এর কোনো নিরাপত্তা-সতর্কতা নেই — HSTS, cookie flags, SSL redirect সব সেট।

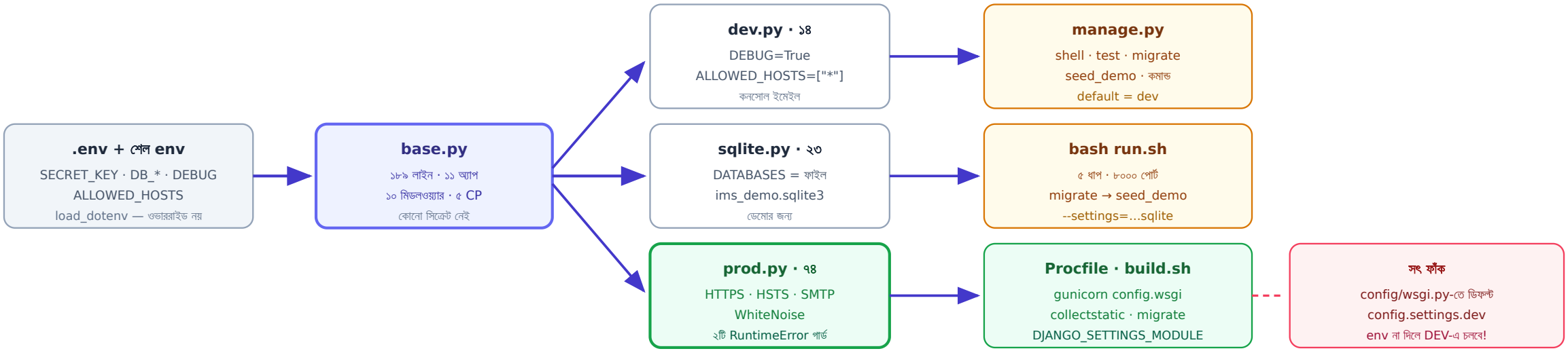
সৎ নোট: এই দুটি গার্ড কোডে লেখা, কিন্তু কোনো টেস্টে নেই — কেউ ভুলে গার্ড সরিয়ে দিলেকেবল CI নয়, কেউ ধরবে না।

শিক্ষকের নোট: দুই গার্ডের যুক্তি এক: "ডিপ্লয় কাজ করছে না" — এর সবচেয়ে সহজ সমাধান সেটিং ঢিলা করা, আর সেটাই সবচেয়ে বিপজ্জনক। এখানে সেই পথ বন্ধ।

এন্ট্রি

কোন দরজা দিয়ে ঢুকলে কোন সেটিং আরম্ভ হয়

একই কোড, চারটি প্রবেশপথ — ভুল প্রবেশপথ মানে ভুল নিরাপত্তা



শিক্ষকের নোট: লাল বাক্সটি পড়ুন শেষে: `wsgi.py`-তে ডিফল্ট dev। অর্থাৎ `DJANGO_SETTINGS_MODULE` সেট না করে gunicorn চালালে ডেমো-সেটিংসে প্রোডাকশন চলবে — এটি প্রকৃত ঝুঁকি, ঠিক করা বাকি।

দরজা

১০টি মিডলওয়্যার, ক্রম অনুযায়ী

উপরে থেকে নিচে — ভেতরে যাওয়ার সময় এই ক্রমে, ফেরার সময় উল্টো

#	মিডলওয়্যার	এক লাইনে কাজ
১–৭	`Security` · `Session` · `Common` · `CSRF` · `Auth` · `Messages` · `XFrame`	Django-র নিজের সাতটি — নিরাপত্তা, সেশন, `request.user`, মেসেজ, ক্লিকজ্যাকিং
৮	`billing.CustomDomainMiddleware`	হোস্ট দেখে প্রতিষ্ঠান চেনে; `Vary: Host` বসায়
৯	`workspaces.CurrentWorkspaceMiddleware`	সেশন/সদস্যতা দেখে `request.workspace` ঠিক করে
১০	`billing.SubscriptionGateMiddleware`	সাবস্ক্রিপশন শেষ হলে লিখতে দেয় না, পড়তে দেয়

প্রোডাকশনে ১১: `prod.py` তালিকার **শুরুতে** `WhiteNoiseMiddleware` যোগ করে — নাহলে স্ট্যাটিক ফাইল পাওয়া যেত না।

ক্রম ভাঙলে: Auth-এর আগে workspace মিডলওয়্যার বসলে `request.user` নেই → সবাই অ্যানোনিমাস, কিন্তু এরর নেই — শুধু ৪০৪।

শিক্ষকের নোট: বোর্ডে তালিকাটি নম্বর দিয়ে লিখুন এবং তীর দিন নিচের দিকে। তারপর জিজ্ঞেস করুন: "উল্টো পথে ফেরার সময় কোনটি আগে চলে?" — সেটাই response-এ হেডার যোগ করার জায়গা।

কেন

workspace ঠিক করা ডেকোরেটরের কাজ নয় — মিডলওয়্যারের

একবার ঠিক করলে ৫২টি ভিউ আর ভাবতে হয় না

প্রশ্ন	ডেকোরেটরে করলে	মিডলওয়্যারে করলে (যা করা হয়েছে)
কত জায়গায় লিখতে হয়	প্রতিটি ভিউতে, অথবা প্রতিটি ভিউতে ডেকোরেটর লাগাতে হয়	একবার, `MIDDLEWARE`-এ
ভুলে যাওয়ার সম্ভাবনা	নতুন ভিউ যোগ করলে ডেকোরেটর বাদ পড়তে পারে	শূন্য — সব রিকোয়েস্ট একই পথে যায়
টেমপ্লেটে পাওয়া যায়?	না, কনটেক্সট আলাদা করে পাঠাতে হয়	হ্যাঁ, `request.workspace` কনটেক্সট প্রসেসরে যায়
প্রতি রিকোয়েস্টে খরচ	একই কোয়েরি, তবু প্রতিটি ভিউতে আলাদা	একটি indexed কোয়েরি, একবার

তবে **ভূমিকা-যাচাই এখনো ডেকোরেটরে** (`@workspace_view(*TEACHING)`) — কারণ "কে কী করতে পারবে" ভিউ-ভিত্তিক সিদ্ধান্ত, আর "কে কোন স্কুলে" রিকোয়েস্ট-ভিত্তিক।

এই ভাগটাই মূল শিক্ষা: **প্রেক্ষাপট (context) মিডলওয়্যারে, অনুমতি (permission) ডেকোরেটরে।**

শিক্ষকের নোট: শিক্ষার্থীদের বলুন নিজের প্রজেক্টে একটি ভিউ বানিয়ে দেখতে — ডেকোরেটর ছাড়া। তারপর জিজ্ঞেস করুন "এটি কোন স্কুলের ডেটা দেখছে?" উত্তর না থাকলেই বুঝবে মিডলওয়্যার কেন দরকার।

কেস ১

CurrentWorkspaceMiddleware — এবং একটি আসল বাগ

সেশন → সদস্যতা → না মিললে ভূমিকা-অনুযায়ী ডিফল্ট

```
ws_id = request.session.get(SESSION_KEY)
qs = Membership.objects.filter(user=request.user,
                               is_active=True)
if ws_id:
    membership = qs.filter(workspace_id=ws_id).first()
    if membership:
        return membership

# ডিফল্ট: সর্বোচ্চ ভূমিকা, তারপর প্রতিষ্ঠান
membership = (qs.order_by(ROLE_PRIORITY,
                          KIND_PRIORITY,
                          "workspace__name").first())
```

অবস্থা	ফল
স্কুলের অ্যাডমিন + অন্যত্র সমন্বয়ক	স্কুলেই খোলে — ভূমিকা-ক্রম দেখে
স্কুলের শিক্ষক + নিজের ব্যক্তিগত স্পেস	স্কুলই আগে — ব্যক্তিগতটি "পিছনের ঘর"
কোনো সদস্যতা নেই	`request.workspace = None` → ভিউ ৪০৪

আসল বাগ: প্রথমে রেজলভিং ছিল "lazy"। কিন্তু `lazy_obj is None` সবসময় False — তাই workspace-হীন ইউজার None-চেক পাশ করে ভেতরে ঢুকে যেত।

সমাধান: রিকোয়েস্টের শুরুতে **সরাসরি** রেজলভ (eager) — প্রতি রিকোয়েস্টে একটি ছোট indexed কোয়েরি।

শিক্ষকের নোট: এই বাগটি শিক্ষার্থীদের সবচেয়ে বেশি কাজে লাগবে — "lazy মানে ফ্রি নয়"। lazy অবজেক্টে `is None` কাজ করে না; শুধু `if obj` ডাকলে সত্য-মিথ্যা বোঝা যায়। এটাই ছিল আসল ফাঁদ।

কেস ২

CustomDomainMiddleware — হোস্টই প্রতিষ্ঠানের পরিচয়

school.example.com → সেই স্কুলের লোগো ও নাম, লগইন পাতাতেই

```
host = request.get_host().split(":")[0].lower()
branding = (WorkspaceBranding.objects
    .select_related("workspace")
    .filter(custom_domain__iexact=host,
            domain_verified=True,
            workspace__is_active=True)
    .first())

if branding:
    request.domain_workspace = branding.workspace
    if host not in settings.ALLOWED_HOSTS:
        settings.ALLOWED_HOSTS += [host]
```

বিবরণ	কেন এভাবে
`domain_verified=True`	যাচাই না হলে ডোমেইন ডাকনাম মাত্র — টাইপ করা হোস্ট বিশ্বাস নয়
`iexact`	DNS নাম case-insensitive; `Example.com` = `example.com`
`ALLOWED_HOSTS`-এ যোগ	ডোমেইনটি ব্যবহারকারীর নিজের, সার্ভার-তালিকায় আগে থেকে নেই
`patch_vary_headers(["Host"])`	একই URL, ভিন্ন স্কুল — ক্যাশে গুলিয়ে ফেলতে পারে না
`OperationalError` ধরা	মাইগ্রেশনের আগে টেবিল না থাকলেও সাইট চলে

প্রমাণ: `test_domain_cannot_be_stolen_by_a_second_institution` · `test_unverified_domain_is_ignored` · `test_verified_domain_shows_that_institution_on_the_login_page` — তিনটি টেস্ট এই মিডলওয়্যারের তিনটি নিয়ম।

শিক্ষকের নোট: `Vary: Host` বাদ দিলে কী হয় — জিজ্ঞেস করুন। উত্তর: এক স্কুলের লগইন পাতা ক্যাশে হয়ে আরেক স্কুলের ব্যবহারকারী সেটাই দেখতে পারে। ক্যাশের বাগ চোখে দেখা যায় না, তাই নিয়মটা লিখে রাখা।

কেস ৩

SubscriptionGateMiddleware — লিখা বন্ধ, ডেটা অটুট

পড়া কখনো বন্ধ হয় না; মুছে ফেলা কখনোই হয় না

নিয়ম	বাস্তবে
`SAFE_METHODS`	GET/HEAD/OPTIONS/TRACE কখনো ব্লক হয় না
`ALLOW_PREFIXES`	`/accounts/` `/renew/` `/platform/` `/static/` `/media/` `/admin/` `/healthz` `/workspaces/`
ORG_ADMIN	`/renew/?blocked=1`-এ পাঠানো হয়
অন্য সবাই	ড্যাশবোর্ডে ফেরত + বার্তা: "আপনার কাজ হারায়নি"

প্রথমে ভুল ছিল: মিডলওয়্যার সেশনের workspace দেখত, URL-এর নয়। এক শিক্ষক নিজের ব্যক্তিগত স্পেসে থাকা অবস্থায় মেয়াদোত্তীর্ণ স্কুলে POST করলে গেট পাশ করে যেত। সমাধান: `_workspace_in_url()` — পথ থেকে slug পড়া।

```
if request.method in SAFE_METHODS:
    return self.get_response(request)

workspace = (self._workspace_in_url(request)
             or getattr(request, "workspace", None))
if workspace is None or any(
        path.startswith(p) for p in ALLOW_PREFIXES):
    return self.get_response(request)

if is_writable(workspace):
    return self.get_response(request)
# ... রিনিউ পাতায় পাঠানো
```

সুপারইউজার ছাড় নেই: প্ল্যাটফর্ম-অ্যাডমিনও যদি ওই স্কুলের অ্যাডমিন হন, তবু লিখতে পারবেন না — কেবল `/platform/` খোলা।

শিক্ষকের নোট: এখানে দুটি নীতি একসাথে: (১) বিলিং মানে ডেটা জিম্মি নয়, (২) অনুমতি URL দেখে, সেশন দেখে নয়। দ্বিতীয়টি না থাকলে এক স্কুল অন্য স্কুলের মেয়াদে ব্লক হয়ে যেত।

ঠিকানা

২৫৩টি URL — কে কোথায় থাকে

`get_resolver()` দিয়ে গোনা · admin ১৬৭ · অ্যাপ ৮৫ · media ১

নেমস্পেস	রুট	উদাহরণ
`billing`	১৮	pricing · renew · console
`academics`	১৩	class_list · term_create
`lessons`	১৩	syllabus · homework_detail
`accounts`	৯	login · signup · password_reset
`assessments`	৯	exam_detail · marksheet
`students` · `reports`	৭ · ৭	student_list · export_students
`attendance`	৪	take · register · summary
`workspaces` · `audit` · `dashboard`	২ · ২ · ১	switch · log_list · home

১১টি `include()`, সবই `""`-এ: `config/urls.py`-তে প্রতিটি অ্যাপের নিজের ফাইল, কিন্তু prefix নেই — পথের প্রথম অংশ প্রতিটি অ্যাপ নিজেই ঠিক করে।

ফল: কে কোন পথ দখল করছে, তা এক নজরে দেখতে `config/urls.py` যথেষ্ট নয় — ১১টি ফাইল খুলতে হয়। এটি জেনে রাখা দুর্বলতা (`docs/url-map.md` নেই)।

সৎ ফাঁক: ১৬৭টি admin URL কোনো ডিজাইন নয় — Django-র admin নিজেই বানায়। তাই "৮৫টি রুট" বলাটাই সৎ।

শিক্ষকের নোট: জিজ্ঞেস করুন: "একই পথ দুই অ্যাপে লিখলে কী হয়?" উত্তর: আগে `include` করা অ্যাপটি জেতে — তাই ক্রমও একটা সিদ্ধান্ত।

নাম

প্রতিটি ঠিকানার একটি নাম — `reverse()`

টেমপ্লেটে `{% url %}`, কোডে `reverse()`, কখনো হাতে লেখা পথ নয়

```
# apps/attendance/urls.py
app_name = "attendance"
urlpatterns = [
    path("w//attendance/",
         views.take, name="take"),
    path("w//attendance/summary/",
         views.summary, name="summary"),
]
```

প্রতিটি ভেতরের পথেই `w/<slug>/` — অর্থাৎ প্রতিটি রুট নিজেই বলে "কোন স্কুল"।

যেখানে	কীভাবে
টেমপ্লেট	`{% url 'attendance:take' slug=ws.slug %}`
ভিউ	`redirect("attendance:summary", slug=…)`
রিডাইরেক্ট	`reverse("billing:renew", kwargs=…)`
মিডলওয়্যার	`reverse(...)` — হার্ডকোড নয়, কারণ গেট সঠিক ঠিকানায় পাঠায়

লাভ: পথ বদলালে এক জায়গায় বদলায় — ভাঙা লিংকের সন্ধানে ৭৫ টেমপ্লেট ঘুরতে হয় না।

কিন্তু: `w/<slug>/` হাতে লেখা প্রতিটি ভিউতে — অর্থাৎ স্কুল-সহ পথ আর স্কুল-ছাড়া পথ একসাথে আছে (`/platform/`, `/accounts/`, `/pricing/`)। কেন, সেটি পরের স্লাইড।

শিক্ষকের নোট: একটি ভাঙা লিংকের গল্প বলুন: URL হাতে লেখা থাকলে এক অক্ষর বদলালেই ৪০৪, আর টেস্ট না থাকলে ধরা পড়ে ব্যবহারকারীর কাছে। `reverse()` সেই ভুলটাই অসম্ভব করে।

সাজ

৫২টি ভিউ একই সাজে — `@workspace_view`

প্রতিটি workspace-ভিউতে তিনটি প্রশ্ন, একটাই উত্তরদাতা

```
@workspace_view(*TEACHING)
def take(request, slug):
    ...

# decorator যা করে:
# ১. লগইন নেই → /accounts/login/?next=...
# ২. ওই স্কুলে সদস্য নয় → 404
# ৩. ভূমিকা অনুমোদিত নয় → 403
# ৪. পাশ হলে request.workspace + membership
```

ভূমিকা-সেট	কারা
`TEACHING`	ORG_ADMIN · COORDINATOR · TEACHER (৩২টি ভিউ)
`ALL_ROLES`	সবাই + STUDENT (৮টি ভিউ)
`OWNERS`	ORG_ADMIN (১১টি ভিউ)
`MANAGERS`	ORG_ADMIN · COORDINATOR (১টি)

তিনটি স্তর: সদস্যতা (404) → ভূমিকা (403) → বস্তু-স্তরের অনুমতি, যেমন `teacher_can_access_class()`। তৃতীয়টি ছাড়া একজন শিক্ষক অন্য ক্লাসের নম্বর দেখতে পারতেন।

কেন ৪০৪ আর ৪০৩ আলাদা: সদস্য না হলে "এই স্কুল আছে কি না" জানানোই তথ্য ফাঁস; তাই ৪০৪। সদস্য কিন্তু ভূমিকা ভুল হলে ৪০৩ — কেন না, সেটা বলা নিরাপদ।

শিক্ষকের নোট: জিজ্ঞেস করুন: "একজন ছাত্র `attendance:take` খুললে কী হবে?" — ৪০৩, কারণ সে সদস্য (৪০৪ নয়) কিন্তু ভূমিকা TEACHING নয়।

কনটেক্সট

প্রতি পাতায় যে ৫টি জিনিস নিজে থেকেই আসে

৩টি Django-র, ২টি নিজস্ব — কোন ভিউ আলাদা করে পাঠায় না

প্রসেসর	যা দেয়	কোথায় কাজে লাগে
`request`	`request`	টেমপ্লেটে পথ, হোস্ট, মিডলওয়্যারের হাতের তথ্য
`auth`	`user` · `perms`	`{% if user.is_authenticated %}`
`messages`	`messages`	ফ্ল্যাশ বার্তা — ৫টি শেল-টেমপ্লেটে একই ব্লক
`workspace_context`	`nav_groups` · `nav_role` · `active_workspace` · `my_memberships`	সাইডবারের মেনু — ভূমিকা অনুযায়ী আলাদা
`branding_context`	`brand` · `product` · `subscription` · `platform_mode`	লোগো, রঙ, পণ্যের নাম, বিলিং ব্যানার

brand একটি dict, মডেল নয়: নতুন স্কুলের ব্র্যান্ডিং সারি না থাকলেও টেমপ্লেট কখনো ক্র্যাশ করবে না — ডিফল্ট পণ্য-নাম দেখাবে।

দাম: এই দুই প্রসেসর প্রতি রিকোয়েস্টে একাধিক কোয়েরি চালায় (সদস্যতা, ব্র্যান্ডিং, সাবস্ক্রিপশন) — লগইন-পাতায়ও চলে, যদিও সেখানে দরকার নেই।

শিক্ষকের নোট: শিক্ষার্থীদের জিজ্ঞেস করুন: "সাইডবারের মেনু কোন ভিউ পাঠায়?" উত্তর — কেউ না। এটাই কনটেক্সট প্রসেসরের লাভ: ৫২টি ভিউতে একই কোড লিখতে হয় না।

পর্দা

৭৫টি টেমপ্লেট — সব এক জায়গায়

`templates/` · অ্যাপের ভেতরে শূন্য টেমপ্লেট · `APP_DIRS=True` তবু ব্যবহার হয়নি

ফোল্ডার	কী
`templates/app_base.html`	অ্যাপের শেল (সাইডবার + টপবার)
`templates/<app>/…`	প্রতি অ্যাপের পাতা
`templates/partials/`	পুনর্ব্যবহারযোগ্য টুকরো
`templates/platform/`	প্ল্যাটফর্ম কনসোল
`templates/public/`	মার্কেটিং পাতা (pricing, signup)
`templates/registration/`	লগইন, পাসওয়ার্ড রিসেট

কেন কেন্দ্রীভূত: ১১টি অ্যাপে টেমপ্লেট ছড়িয়ে দিলে `{% extends "app_base.html" %}` খুঁজতে ১১টি ফোল্ডার লাগত; আর একই নামের দুই টেমপ্লেট সংঘর্ষ করত।

দাম: অ্যাপ আলাদা করে সরানো এখন কঠিন — টেমপ্লেট তার বাইরে থাকে। মডুলারিটি এখানে ফোল্ডার-স্তরে, প্যাকেজ-স্তরে নয়।

প্রমাণ: `find apps -name "*.html"` → **০** · `find templates -name "*.html"` → **৭৫**

শিক্ষকের নোট: দুই পদ্ধতিরই দাম আছে — অ্যাপ-ভেতরে টেমপ্লেট মানে অ্যাপ সরানো সহজ, কেন্দ্রীভূত মানে ভাগ করা শেল সহজ। এখানে দ্বিতীয়টি বেছেছেন, কারণ শেল-ই বড় কাজ।

স্ট্যাটিক

৫টি ফাইল, ৫৩৪ KB — এবং একটি CDN-ও নেই

`static/vendor/` — Bootstrap আর আইকন নিজের ভেতরে রাখা

ফাইল	আকার
`vendor/bootstrap.min.css`	২২৭ KB
`vendor/bootstrap-icons.woff2`	১২৭ KB
`vendor/bootstrap-icons.css`	৮৪ KB
`vendor/bootstrap.bundle.min.js`	৭৯ KB
`css/ims.css`	৮ KB

কেন CDN নয়: স্কুলের নেটওয়ার্ক প্রায়ই বন্ধ-নিয়ন্ত্রিত; CDN ব্লক হলে পর্দা ভেঙে পড়ে। নিজের ফাইল মানে নিজের নিয়ন্ত্রণ।

দাম: আপডেট নিজে টানতে হয় — Bootstrap-এর নিরাপত্তা-প্যাচ স্বয়ংক্রিয়ভাবে আসে না। ভার্সনও কোথাও লেখা নেই।

প্রোডাকশনে: `collectstatic` → `staticfiles/` ; WhiteNoise manifest + ৩০ দিন ক্যাশ; ফাইলের নামে হ্যাশ, তাই পুরনো ক্যাশ কখনো ভুল ফাইল দেয় না।

সৎ ফাঁক: `staticfiles/` ফোল্ডার `.gitignore`-এ আছে, কিন্তু কোন Bootstrap ভার্সন, তা কোথাও লেখা নেই — আপডেট করতে গেলে প্রথমেই অনুমান করতে হবে।

শিক্ষকের নোট: "CDN মানে দ্রুত" — এই ধারণাটি প্রশ্ন করুন। বাংলাদেশের স্কুল-নেটওয়ার্কে CDN ব্লক হওয়ার ঘটনা বাস্তব; ৫৩৪ KB নিজের সার্ভার থেকে দিলে ৫০ms বেশি লাগে, কিন্তু পর্দা কখনো ভাঙে না।

সেশন

সেশন ও কুকি — ৮ ঘণ্টা, আর ৩টি পতাকা

"কে লগইন করা আছে" আর "কোন স্কুলে আছে" — দুটোই সেশনে

সেটিং	মান
`SESSION_COOKIE_AGE`	৮ ঘণ্টা (এক কর্মদিবস)
`SESSION_SAVE_EVERY_REQUEST`	`True` — ব্যবহারে মেয়াদ বাড়ে
`SESSION_EXPIRE_AT_BROWSER_CLOSE`	`False`
প্রোডাকশনে	`Secure` · `HttpOnly` · `SameSite=Lax`

সেশন-কী: `active_workspace_id` — কোন স্কুলে আছেন, সেটাই সেশনে থাকে।

একটি আসল ফাঁদ: সেশনের workspace আর URL-এর workspace আলাদা হতে পারে। এই পার্থক্যই একবার SubscriptionGate-এ বাগ তৈরি করেছিল (আগের স্লাইড)।

কেন ৮ ঘণ্টা: স্কুলে একটি শিফট ৬–৭ ঘণ্টা; মাঝপথে লগআউট হলে শিক্ষক হাজিরার মাঝখানে আটকে যাবেন। ৩০ দিন রাখলে শেয়ার্ড কম্পিউটারে ঝুঁকি।

CSRF: আলাদা কুকি, আলাদা টোকেন; ব্যর্থ হলে নিজের ভিউ `apps.core.views.csrf_failure` — কুৎসিত ডিফল্ট পাতা নয়।

শিক্ষকের নোট: শেয়ার্ড কম্পিউটারের কথা বলুন — স্কুলের ল্যাবে একজন লগআউট না করে চলে গেলে পরেরজন তার নামে হাজিরা দিতে পারে। এটাই সেশন-মেয়াদের আসল প্রশ্ন, প্রযুক্তিগত নয়।

ইমেইল

একই কোড, দুই ব্যাকএন্ড — কনসোল নাকি SMTP

ডেমোতে ইমেইল টার্মিনালে, প্রোডাকশনে আসল সার্ভারে

পরিবেশ	ব্যাকএন্ড ও ফল
dev · sqlite	`console.EmailBackend` — চিঠি টার্মিনালে ছাপা হয়, কেউ পায় না
prod	`smtp.EmailBackend` · পোর্ট ৫৮৭ · STARTTLS
প্রেরক	`DEFAULT_FROM_EMAIL` — পণ্যের নাম থেকে, হার্ডকোড নয়

কোন পথে ইমেইল যায়: পাসওয়ার্ড রিসেট · স্কুল-অনুরোধের স্বীকৃতি · আমন্ত্রণ।

প্রমাণ: `test_email_is_sent_and_link_sets_a_new_password` — মেইল বেরোয়, লিংকের টোকেন কাজ করে, পাসওয়ার্ড বদলায়। তিন ধাপ এক টেস্টে।

সৎ ফাঁক: SMTP পাঠানোর কোনো রিট্রাই নেই — পাঠানো ব্যর্থ হলে ইমেইল হারায়, ব্যবহারকারী শুধু "লিংক আসেনি" বলেন। Celery/কিউ ছাড়া এটাই সীমা।

টেস্টে আসল মেইল যায় না: Django-র `mail.outbox` ব্যবহার হয় — ইন্টারনেট ছাড়াই টেস্ট চলে।

শিক্ষকের নোট: কনসোল ব্যাকএন্ড শিক্ষার্থীদের প্রিয় — কারণ লিংকটা সরাসরি দেখতে পান। বলুন: "ডেমোতে ইমেইল মানে টার্মিনাল; প্রোডাকশনে ইমেইল মানে দায়িত্ব।"

কমান্ড

৪টি ম্যানেজমেন্ট কমান্ড, ৪০৩ লাইন

যে কাজ একবার করতে হয়, তার জন্য ভিউ নয় — কমান্ড

কমান্ড	লাইন	কী করে
`seed_demo`	২৮১	demo ডেটা: স্কুল, ৪০ ছাত্র, শ্রেণি, হাজিরা, পরীক্ষা — `run.sh` নিজে ডাকে
`billing_report`	৫১	কোন স্কুল কত seats ব্যবহার করছে, কত টাকা বাকি — টার্মিনালে ছাপা
`sync_domains`	৩৭	যাচাই করা কাস্টম ডোমেইন নিয়ে `ALLOWED_HOSTS`-এর তালিকা ছাপে/মেলায়
`seed_plans`	৩৪	তিনটি প্ল্যান তৈরি/আপডেট — মাইগ্রেশন ছাড়াই পুনরায় চালানো যায়

কেন ভিউ নয়: এই কাজের জন্য লগইন, সেশন বা CSRF লাগে না — আর ভুল করে কোনো ব্রাউজার থেকে চালানোও উচিত নয়।

সবচেয়ে বড়িটি `seed_demo` : ২৮১ লাইন — ডেমোর পুরো গল্প এখানেই লেখা; তাই ডেমো রিসেট করা (`--reset`) মানে এক কমান্ডে নতুন গল্প।

সৎ ফাঁক: `seed_demo`-র কোনো টেস্ট নেই। কেউ ভেঙে দিলে টের পাবেন কেবল ডেমো চালিয়ে।

শিক্ষকের নোট: প্রশ্ন করুন: "ডেটা বানানোর কাজটা ভিউতে রাখলে কী সমস্যা?" উত্তর: লগইন লাগত, আর ভুল করে প্রোডাকশনে কেউ ক্লিক করলে ডেমো-ডেটা ঢুকে যেত।

সংযোগ

ডেটাবেজ-সংযোগ: একবার খুলে রাখা, ৬০ সেকেন্ড

`CONN_MAX_AGE=60` · `CONN_HEALTH_CHECKS=True` · psycopg 3

```
"ENGINE": "django.db.backends.postgresql",
"NAME": env("DB_NAME", "ims_dev"),
"USER": env("DB_USER", "ims_user"),
"HOST": env("DB_HOST", "127.0.0.1"),
"PORT": env("DB_PORT", "5432"),
"CONN_MAX_AGE": 60,
"CONN_HEALTH_CHECKS": True,   # prod-এ
```

সেটিং	কেন
CONN_MAX_AGE ৬০	প্রতি রিকোয়েস্টে নতুন সংযোগ নয় — হ্যান্ডশেক বাঁচে
HEALTH_CHECKS	পুরনো সংযোগ মৃত হলে ব্যবহারের আগেই ধরা পড়ে
gunicorn workers ৩	worker প্রতি আলাদা সংযোগ-পুল; ৪টি worker মানে ৪০+ সংযোগ

সীমা: পুলিং লাইব্রেরি (pgbouncer) নেই — worker বাড়ালে PostgreSQL-এর `max_connections` -এ আটকে যাবে।

তিনটি পরিবেশ, একই কোড: PostgreSQL (`dev` / `prod`) · SQLite (`sqlite` — শুধু ডেমো)। SQLite-তে `CONN_MAX_AGE` -এর অর্থ আলাদা, তাই ডেমোর গতি আর প্রোডের গতি কখনো এক নয়।

শিক্ষকের নোট: শিক্ষার্থীদের জিজ্ঞেস করুন: "worker ৩ থেকে ৩০ করলে কী ভাঙবে?" উত্তর: সংযোগের সংখ্যা — আর সেটি ডেটাবেজ-সার্ভারেই ধরা পড়ে, অ্যাপে নয়।

তার

তিনটি স্ক্রিপ্ট যা সব জোড়া লাগায়

`run.sh` (ডেমো) · `build.sh` (বিল্ড) · `Procfile` (চালু)

run.sh — ৫ ধাপ

```
১. python3 আছে? সংস্করণ ≥ 3.12?
২. .venv না থাকলে বানাও
৩. pip install -r requirements.txt
৪. migrate + seed_demo (sqlite)
৫. runserver 127.0.0.1:8000
```

প্রতিটি ধাপে স্পষ্ট বার্তা — "কেন ব্যর্থ হলো" না বলে থামে না।

build.sh

```
pip install -r requirements-prod.txt
python manage.py collectstatic --noinput
python manage.py migrate --noinput
```

Procfile

```
release: migrate --noinput
web: gunicorn config.wsgi --workers 3 --timeout 60
```

প্রশ্ন	উত্তর
কেন migrate দুই জায়গায়?	`build.sh`-এ বিল্ডের সময়, `release`-এ চালুর সময় — কোনও PaaS একটিকে ডাকে, অন্যরা অন্যটি
কেন `--noinput`?	স্বয়ংক্রিয় পরিবেশে প্রশ্ন করার কেউ নেই; প্রশ্ন এলে ডিপ্লয় ঝুলে থাকে
কেন ৩ worker?	ডেমোর লোডে যথেষ্ট; বাড়ানো হিসাব করে, অনুমানে নয়

শিক্ষকের নোট: "একই কাজ দুই জায়গায়" দেখে শিক্ষার্থীরা ভুল ভাবে — এখানে ইচ্ছাকৃত, কারণ ভিন্ন প্ল্যাটফর্ম ভিন্ন ধাপ ডাকে। বলুন: পুনরাবৃত্তি সবসময় দোষ নয়, অজানা ক্রম থেকেও খারাপ।

সততা

সংযোগ-স্তরের ৮টি ফাঁক — প্রতিটির একটি সংশোধন

লুকানো নেই, কারণ এগুলোই পরের কাজের তালিকা

ফাঁক	আজকের ঝুঁকি	সংশোধন
`/healthz` এন্ডপয়েন্ট নেই	গেটে অনুমতি আছে, ঠিকানা নেই — মনিটরিং কী দেখবে?	একটি ভিউ যোগ, DB-চেক সহ
`DATABASE_URL` অপ্রয়োগ	PaaS-এর ডিফল্ট চলবে না; `dj-database-url` অচল প্যাকেজ	`prod.py`-তে URL পার্স করা
`wsgi.py` ডিফল্ট `dev`	env ভুলে গেলে প্রোডাকশন ডেমো-সেটিংসে	ডিফল্ট `prod`, ওভাররাইড env দিয়ে
`.env.example`-এ `DB_*` নেই	নতুন কেউ সেটআপে অনুমান করে	৫টি কী যোগ
ক্যাশ স্তর নেই	প্রতি রিকোয়েস্টে কনটেক্সট-কোয়েরি	মেপে দরকার হলে Redis
ASGI ঘোষিত, WSGI ব্যবহৃত	দুই এন্ট্রি পয়েন্ট, একটি অব্যবহৃত	ASGI সরানো, বা ব্যবহার শুরু
prod-এর ২টি গার্ড টেস্টহীন	গার্ড সরিয়ে দিলে কেউ ধরবে না	দুটি ছোট টেস্ট
vendor ভার্সন অনির্দিষ্ট	Bootstrap আপডেটে ভাঙার ঝুঁকি অজানা	ভার্সন ফাইলে লিখে রাখা

প্রথম চারটি একদিনের কাজ — তবু ডেমো ঠিক চলছে, তাই কেউ করে না। **এটাই প্রযুক্তিগত ঋণের আসল চেহারা:** ভাঙা নয়, শুধু ঝুঁকিপূর্ণ।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন এই তালিকা থেকে একটি বেছে নিয়ে ঠিক করতে — ৩০ মিনিটে সম্ভব। যে কাজ নিজে করেছে, সেটাই সত্যিকারের শেখা।

৩

D3 — নিরাপত্তা

রিকোয়েস্ট ভেতরে ঢুকল, এখন কে কী দেখবে আর কী করতে পারবে · ২২ স্লাইড